List of question of slide - 8

1. what's the problem on 2 way handshaking in connection establishment under" transport layer mention with example (P: 9-10)

❌Problem with 2-Way Handshaking in Connection Establishment (Transport Layer)

⚠️What's the issue?

While **2-way handshaking** (just sending a **CONNECTION REQUEST** and receiving a **CONNECTION ACCEPTED**) might *seem* enough to establish a connection, it's **not reliable** in real-world networks where packets can be:

- **Lost**
- **Duplicated**
- **Delayed (stored temporarily)**

This opens the door to **serious data integrity and security issues**.

🧨Problem Example: The Duplicate Packet Nightmare

Let's say:

- You (the user) **connect to your bank** using a network.
- You send a **CONNECTION REQUEST**, and the bank **accepts** it.
- You instruct the bank to **transfer money** to someone (possibly shady).
- After the transaction, the **connection is closed**.

Now here's the twist:

- The network **stores all your packets** (request, instructions, release).
- Later, due to a malfunction, it **resends** the entire sequence to the bank.
- From the bank's point of view:
  - A new CONNECTION REQUEST came in.
  - It receives valid-looking instructions (to transfer money).

- So it **executes the transaction again**, **assuming it's new**.

🚫 Why 2-Way Handshake Fails

- **No verification of freshness**: There's no way for the receiver (bank) to tell whether a request is new or a delayed duplicate.
- **No sequence number check**: Without a 3rd step confirming receipt and uniqueness, it's **impossible to detect replayed requests**.
- **Result**: **Unintended duplicate actions** (e.g., money transferred twice).

✅ Solution:

3-Way Handshaking

The **TCP 3-way handshake** solves this by adding:

1. **SYN**: Client → Server (connection request, with sequence number)
2. **SYN-ACK**: Server → Client (acknowledges request, sends its own sequence)
3. **ACK**: Client → Server (acknowledges the server's SYN)

This allows:

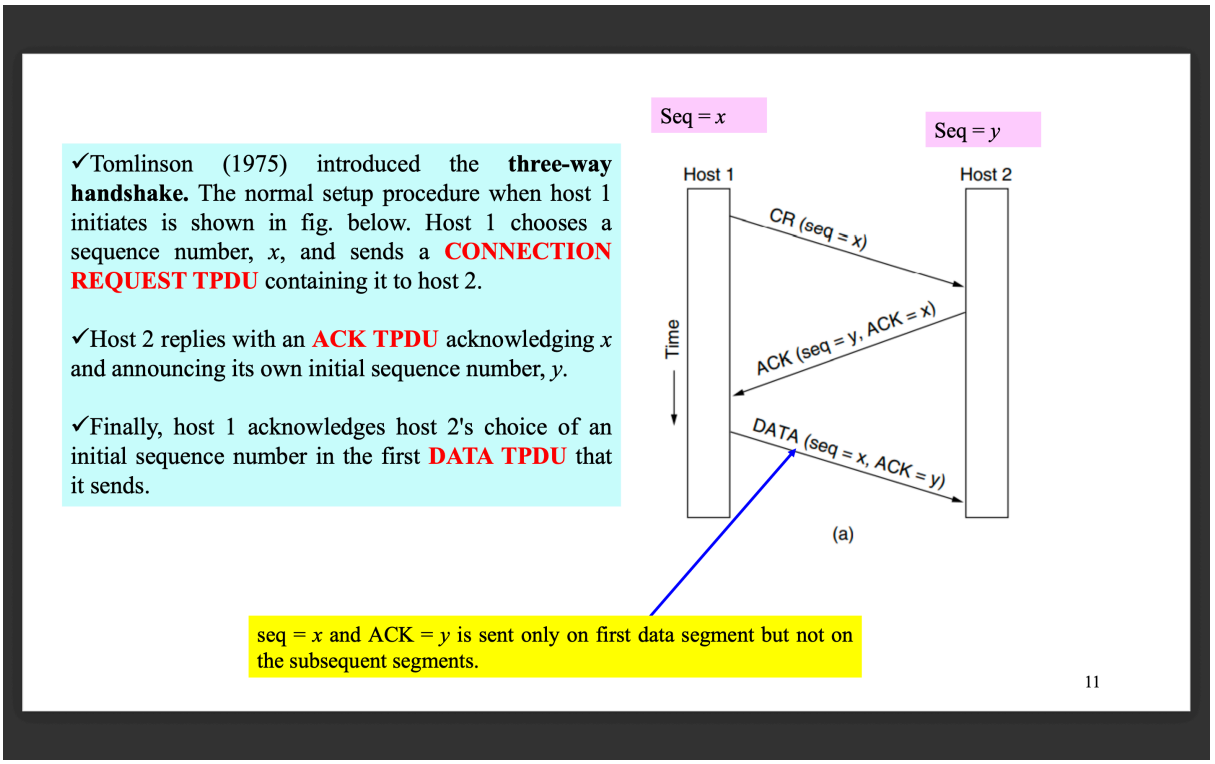
- Mutual verification
- Use of **sequence numbers** to detect duplicates
- Safer, more reliable connection setup

🧾 Summary

Aspect	2-Way Handshake	3-Way Handshake
Simplicity	Simple but unsafe	Slightly more complex
Handles delayed/duplicate packets	❌ No	✅ Yes
Safe for sensitive operations (e.g., bank transfers)	❌ No	✅ Yes
Common use	Rare	Used in TCP

Would you like a diagram comparing 2

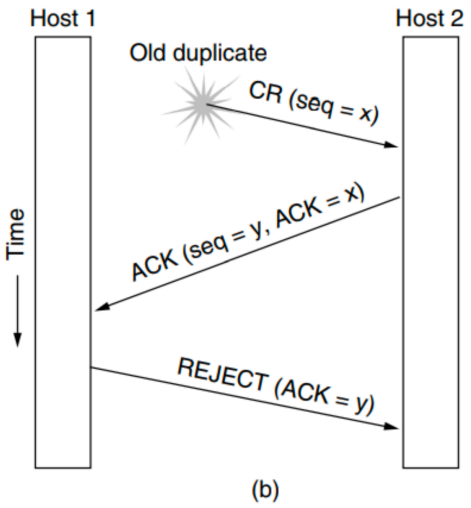
2. Draw the basic of 3 way handshaking (p - 11 fig)



✓Now let us see how the three-way handshake works in the presence of **delayed duplicate control TPDUs**.

✓In Fig. below, the first TPDU is a delayed duplicate CONNECTION REQUEST from an old connection. This TPDU arrives at host 2 without host 1's knowledge. Host 2 reacts to this TPDU by sending host 1 an ACK TPDU, in effect asking for verification that host 1 was indeed trying to set up a new connection.

✓When host 1 rejects host 2's attempt to establish a connection, host 2 realizes that it was tricked by a delayed duplicate and abandons the connection. In this way, a delayed duplicate does no damage.



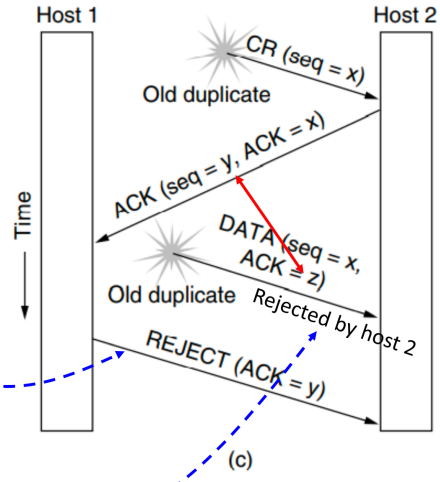
12

3. Old duplicate CR and old duplicate Data packet are rejected, under 3 way hand shaking (P: 13 only fig)

The worst case is when **both a delayed CONNECTION REQUEST (CR)** and **an ACK are floating around in the subnet**. This case is shown in Fig. below.

✓As in the previous example, host 2 gets a delayed **CONNECTION REQUEST (CR)** and replies to it using *y* as the initial sequence number. This CR will be rejected by Host-1.

✓When the second delayed TPDU:
DATA(seq = x, ACK = z) arrives at host 2, the fact that z has been acknowledged rather than y tells host 2 that this, is an old duplicate.



13

4. Compare Symmetric and asymmetric release to connection (p: 14 second para)

🔄 Comparison: Symmetric vs Asymmetric Release in Connection Termination

Feature	Symmetric Release	Asymmetric Release
🔁 **Direction Handling**	Treats connection as **two separate** unidirectional channels. Each must be closed **individually**.	Treats connection as a **single unit**. When **one side ends**, the **entire connection closes**.
⚙️ **How It Works**	Both sides must **explicitly agree** to terminate. Each side sends a **DISCONNECT** message.	One side sends a **DISCONNECT**, and the connection is terminated **immediately**.
📞 **Real-life Example**	Like **messaging apps** (e.g., WhatsApp call), where both users may need to end the call separately.	Like the **telephone system**—when one person hangs up, the line disconnects.
✅ **Use Case**	Preferred in **computer networks** where **orderly and reliable** termination is needed.	Simpler, used in scenarios where **speed** and **simplicity** are more important than full acknowledgment.
🧠 **Risk/Challenge**	More control, but needs **extra coordination**. Risk of one side thinking the connection is open while the other has closed it.	Less control; the other party might not be ready for disconnection. Possible **data loss** if not coordinated.

🧩 Example:

Symmetric Release (TCP-style):

A → B: DISCONNECT
B → A: DISCONNECT
(Both ends agree. Safe closure.)

Asymmetric Release (Telephone-style):

A → B: DISCONNECT
(Connection ends immediately)

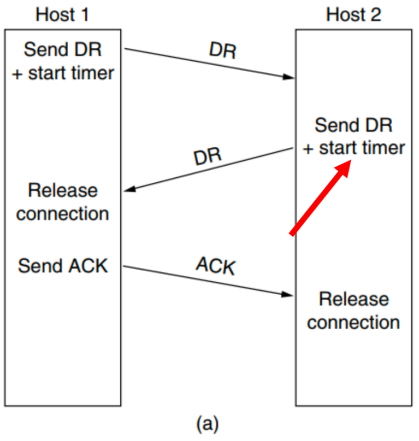
📌 Summary:

- **Symmetric release** = safer, cleaner, but requires both sides to agree.
- **Asymmetric release** = faster, simpler, but riskier if one side isn't done yet.

Would you like a real-world analogy or visual chart to go along with this explanation?

5. show the 3 way hand shake of connection release under TCP (P-18 Only diagram)

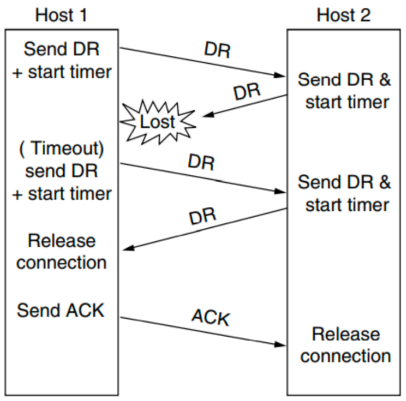
✓We will next consider using a three-way handshake for four scenarios of releasing connection.
✓In Fig. (a), we see the normal case in which one of the users sends a DR (**DISCONNECTION REQUEST**) TPDU to initiate the connection release.
✓When DR arrives, the recipient sends back a DR TPDU, too, and **starts a timer**, just in case its DR is lost.
✓When this DR arrives, the original sender sends back an ACK TPDU and releases the connection.
✓Finally, when the ACK TPDU arrives, the receiver also releases the connection.



18

6. How it tackles lose of DR (Disconnect request) (P: 20-21)

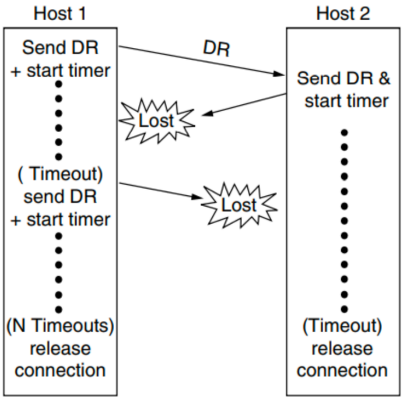
Now consider the case of the second DR being lost. The user initiating the disconnection will not receive the expected response, will time out, and will start all over again. In Fig. (c) we see how this works, assuming that the second time no TPDUs are lost and all TPDUs are delivered correctly and on time.



(c)

20

Our last scenario, Fig. (d), is the same as Fig. (c) except that now we assume all the repeated attempts to retransmit the DR also fail due to lost TPDUs. After N retries, the sender just gives up and releases the connection. Meanwhile, the receiver times out and also exits.



(d)

21

7. What in CWND and RWND (p: 34)

TCP uses **flow control** and **congestion control** mechanisms to efficiently and reliably send data over the network. Two important variables used for this purpose are:

1. CWND (Congestion Window)

- 💡 **Definition:** A TCP state variable that limits the amount of data the sender can transmit **into the network** before receiving an acknowledgment (ACK).
- 📊 **Controlled by:** The **congestion control algorithm** (e.g., slow start, congestion avoidance).
- 📉 **Dynamic Behavior:**
  - Increases when the network is stable (more bandwidth available).
  - Decreases when packet loss or congestion is detected.

---

2. RWND (Receiver Window)

- 💡 **Definition:** A TCP state variable that specifies how much **buffer space is available** at the receiver end.
- 📊 **Controlled by:** The **receiver's application** and **buffer availability**.
- 📭 Receiver uses the **TCP header** to inform the sender of its current rwnd value.

🔪 Effective Transmission Window Size

The sender **must respect both** the network and the receiver's limits:

Actual Window Size= min(CWND, RWND)

🧩 Example:

- CWND = 8 KB (network can handle this much without congestion)
- RWND = 5 KB (receiver only has 5 KB buffer space available)
- ✅ **Sender can only send**: **5 KB** at most before waiting for ACKs.

📌 Summary Table:

Variable	Controlled By	Purpose	Affects
CWND	Sender (congestion control)	Prevent **network overload**	Limits data based on **network capacity**

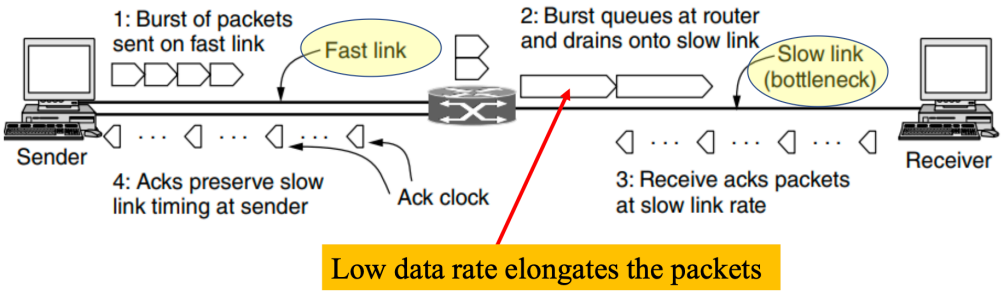
Variable	Controlled By	Purpose	Affects
RWND	Receiver (buffer size)	Prevent **receiver overflow**	Limits data based on **receiver capacity**
min(CWND, RWND)	Combined effect	Ensures reliable, congestion- and flow-controlled transmission	Defines **how much data** sender can send without ACK

Would you like a diagram to visualize how CWND and RWND interact?

8. How ACK clock in acquired (p: 37-39)

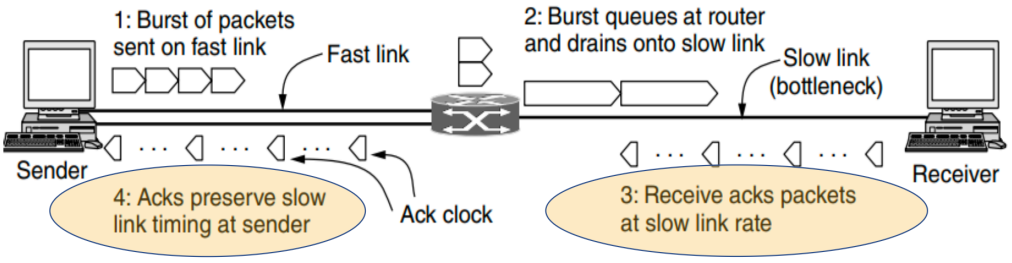
✓ Fig. below shows what happens when a sender on a **fast network** (the 1-Gbps link) sends a small burst of four packets to a receiver on a **slow network** (the 1- Mbps link) i.e. the bottleneck or slowest part of the path.

✓ Initially the four packets travel over the link as quickly as they can be sent by the sender. At the router, they are queued while being sent because it takes longer to send a packet over the slow link than to receive the packet over the fast link.



✓ Eventually the packets get to the receiver, where they are acknowledged. The times for the acknowledgements reflect the times at which the packets arrived at the receiver after crossing the slow link.

✓ These acknowledgements travel over the network and back to the sender they preserve this timing.



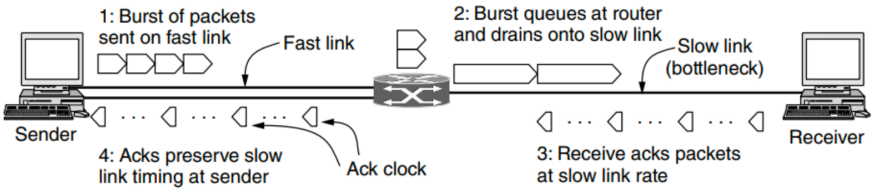
A burst of packets from a sender and the returning ack clock

38

✓ The key observation is this: the acknowledgements return to the sender at about the rate that packets can be sent over the slowest link in the path. This is precisely the rate that the sender wants to use.

✓ If it injects new packets into the network at this rate, they will be sent as fast as the slow link permits, but they will not queue up and congest any router along the path. This timing is known as an **ack clock**. It is an essential part of TCP.

✓ By using an **ack clock**, TCP smoothens out traffic and avoids unnecessary queues at routers.



A burst of packets from a sender and the returning ack clock

39

The **ACK clock is acquired by observing the rate at which acknowledgments return from the receiver**. This rate reflects the capacity of the bottleneck link in the path. TCP uses this timing to regulate the sending rate, ensuring that packets are sent just fast enough to match the network's ability to deliver them —preventing congestion and maintaining flow control.